SAVEETHA SCHOOL OF ENGINEERING

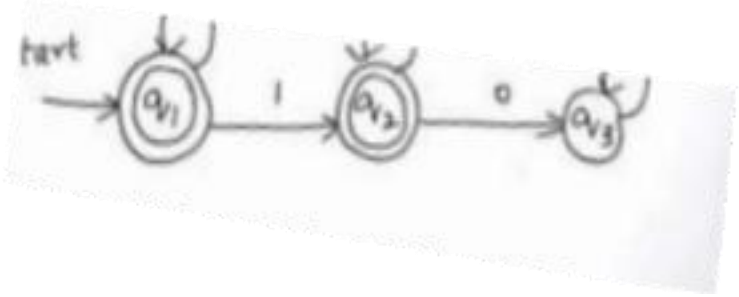
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING

CSA13 – THEORY OF COMPUTATION

ASSIGNMENT - 2

		CO	BTL
1	Write closure properties of regular languages with respect to compliment, kleen star, reversal and substitution	2	4
2	Construct a Non Deterministic Finite automata with ε-transitions to represent the language denoted by the regular expression (a+b)*(ab)*	2	3
3	Give an informal description of the language denoted by the regular expression (0+1)*00	2	3
4	Justify that the language L consisting of all strings with an equal number of a's and b's over the alphabet ΣΣ={a,b} is not regular using pumping lemma .	2	4

5	Apply Arden's Theorem and derive the regular expression for the language represented by the given automata 	2	3

1. Closure Properties of Regular Languages

A language is called regular if it can be represented by a regular expression or accepted by a finite automaton.

Regular languages satisfy the following closure properties:

1. Closure under Complement

If L is a regular language, then its complement is also regular.

Complement contains all strings that do not belong to L.

If L is accepted by a DFA, interchange its final and non-final states to obtain its complement.

Example:

If Σ = ,

L =

Then,

overline{L} = Σ* - L

Therefore, complement is regular.

2. Closure under Kleene Star

If L is a regular language, then its Kleene Star (L*) is also regular.

Kleene Star represents zero or more repetitions of strings in L.

It always includes the empty string (ε).

Example:

If L = ,

L^* = {ε, ab, abab, ababab, …}

Therefore, L* is regular.

3. Closure under Reversal

If L is a regular language, then its reversal (L^R) is also regular.

Reversal means writing each string in the reverse order.

A finite automaton can be constructed to accept the reversed strings.

Example:

If L = ,

L^R = {ba,baa,bba}

Therefore, reversal is regular.

4. Closure under Substitution

If L is a regular language and each symbol is replaced by a regular language, the resulting language is also regular.

Example:

Let L = .

Substitute:

a → 0

b → 11

Then,

L' = {011}

Therefore, the resulting language is regular.

Summary Table

Closure property	**Result**
Complement	Regular

Kleene Star	Regular
Reversal	Regular
Substitution	Regular

Conclusion: Regular languages are closed under complement, Kleene Star, reversal and substitution. This means applying any of these operations to a regular language produces another regular language.

2. Construct an NFA with ε-transitions for the regular expression

(a+b)^*(ab)^*

Step 1: Identify the components

The given regular expression contains two parts:

(a+b)* – Accepts any combination of a and b, including the empty string (ε).

(ab)* – Accepts zero or more repetitions of the string ab.

Step 2: Construct the ε-NFA

Define the following states:

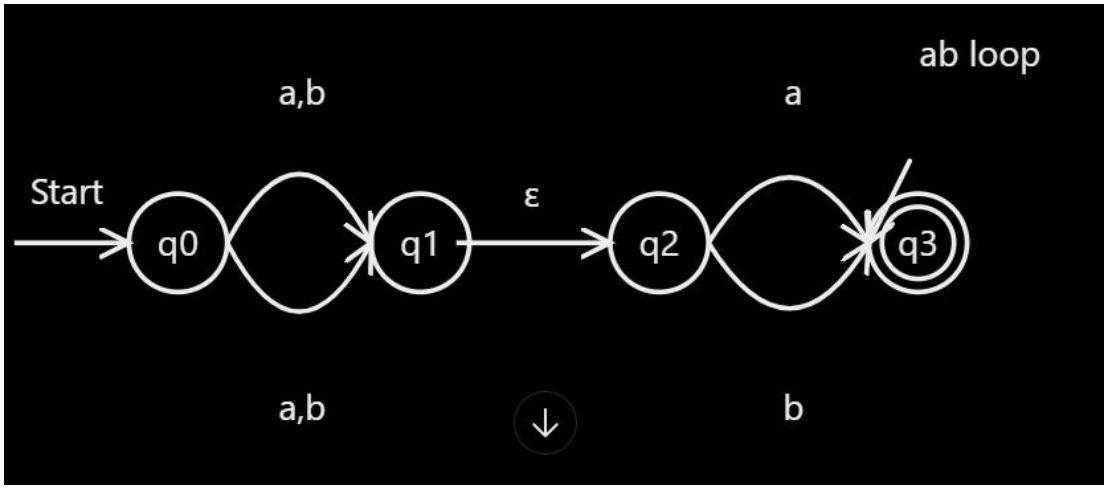
q0 – Initial state

q1 – Final state

q2 – Intermediate state

q3 – Final state

State transition diagram



Conceptual state diagram; use the transition table below for the exact construction.

Step 3: Transition table

Current state	Input a	Input b	ε
q0			
q1		–	
q2	–		–
q3	–	–	–

Step 4: Explanation

q0 loops on both a and b to represent (a+b)*.

From q0, an ε-transition moves to q1.

q1 and q2 represent the repetition of ab.

q1 moves to q2 on input a.

q2 moves to q1 on input b.

q1 and q3 are accepting states.

Step 5: Final answer

Initial state: q0

Final states:

Input alphabet:

Transitions: As shown in the table.

ε-transitions: q0 → q1 and q1 → q3.

Conclusion: The constructed ε-NFA represents the regular expression (a+b)*(ab)* and accepts all strings over {a,b} that can be divided into any combination of a and b, followed by zero or more repetitions of ab.

3. Informal Description of the Language Denoted by the Regular Expression (0+1)*00

Given Regular Expression:

(0+1)*00

Explanation:

(0+1): Represents either 0 or 1.

(0+1)*: Represents any combination of 0s and 1s, including the empty string (ε).

00: Represents two consecutive zeros at the end of the string.

Informal Description:

The regular expression (0+1)*00 represents the set of all binary strings over the alphabet {0,1} that end with two consecutive zeros (00).

Examples

Binary string	**Accepted/Rejected**
00	Accepted
100	Accepted
000	Accepted
1100	Accepted
10100	Accepted
0101	Rejected
101	Rejected
111	Rejected

Conclusion

The language contains all possible combinations of 0s and 1s, provided that every string ends with 00.

L = {00, 100, 000, 1100, 0100, ...}

In simple words: Any binary string ending with two consecutive zeros is accepted by this regular expression.

4. Prove that the language L consisting of all strings with an equal number of a's and b's over Σ = is not regular using the Pumping Lemma.

Given:

L = {w ϵ {a,b}* | n_a(w) = n_b(w)}

Here, the number of a's must be equal to the number of b's.

Step 1: Assume that L is regular

According to the Pumping Lemma, if L is regular, there exists a pumping length p.

Any string w in L with length at least p can be divided into three parts:

w = xyz

Subject to the following conditions:

|xy| <= p

|y |> 0

xy^I z ϵ L, for all i >= 0.

Step 2: Select a string

Consider the string:

W = a^p b^p

This string contains:

p number of a's.

p number of b's.

Therefore, w ϵ L.

Step 3: Divide the string

According to the Pumping Lemma:

w=xyz

Since |xy| <= p, both x and y contain only a's.

Let:

X = a^m

Y = a^k, k>0
Z = a^{p-m-k} b^p

Step 4: Pump the string

Take \(i=0\), which means removing y.

xy^0z = xz
= a^{p-k} b^p

Now:

Number of a's = p − k

Number of b's = p

The number of a's is not equal to the number of b's.

Therefore,

xy^0z not in L

Step 5: Contradiction

This contradicts the Pumping Lemma condition that:

xy^iz ϵ L

for all i > 0.

Hence, our assumption that L is regular is false.

Final Result
L is NOT a regular language.

Conclusion: The language containing strings with an equal number of a's and b's is not regular because it violates the Pumping Lemma.

5. Apply Arden's Theorem and derive the regular expression for the given automata.

Step 1: Identify the states

From the given automaton:

q1 – Initial state and final state.

q2 – Final state.

q3 – Non-final (dead) state.

Transition from q1 to q2 is labelled 1.

Transition from q2 to q3 is labelled 0.

Assuming the self-loop labels are 0 at q1, 1 at q2, and 0,1 at q3.

Step 2: Write the regular equations

Let R1, R2 and R3 represent the regular expressions reaching states q1, q2 and q3 respectively.

For q1:

R1 = ε + R1(0)

For q2:

R2 = R1(1) + R2(1)

For q3:

R3 = R2(0) + R3(0+1)

Step 3: Apply Arden's Theorem

Arden's Theorem:

R = Q + RP --> R = QP*

For q1:

R1 = ε + R1(0)

Applying Arden's theorem:

R1 = ε(0)*
R1 =0^*

For q2:

R2 = R1(1) + R2(1)

Substitute R1:

R2 = 0^*1 + R2(1)

Applying Arden's theorem:

R2 = 0*11*

Step 4: Find the final regular expression

The final states are q1 and q2.

Therefore,

R = R1 + R2

Substitute the values:

R = 0* + 0*11*

Taking 0* common:

R = 0^*(ε + 11*)

Simplifying:

R = 0*1*

Final Answer

0*1*

Conclusion

The regular expression is 01, which represents all strings containing zero or more 0s followed by zero or more 1s.